ALLIANCE UNIVERSITY

Private University Estd. in Karnataka State by Act No . 34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

School of Advanced Computing

Department of Computer Science and Engineering

B. Tech. CSE, DA-B

Project Report

Project Title:
CREDIT CARD FRAUD DETECTION

Project Type:
MACHINE LEARNING

Year:
2025

Author:
SIDDAREDDY GARI MYTHILI REDDY

Table of Contents

s.no	Description	Page.no
1	Abstract	3
2	Introduction	4
3	Objectives	5
4	Problem Statement	6
5	Methodology	7
6	Work Load Matrix	9
7	Literature Review	10
8	Data Collection	11
9	Data Preprocessing	13
10	Model Development	15
11	Model Evaluation	17
12	Conclusion	19
13	References	20

1. Abstract

Credit card fraud has become one of the most significant challenges in the financial sector due to the increase in online transactions and digital payment systems. This project aims to detect fraudulent credit card transactions using machine learning techniques. By analyzing patterns and behaviors within transaction data, the system can identify suspicious activities in real-time.

The dataset used includes both legitimate and fraudulent transactions, with features such as transaction amount, time, and anonymized customer data. Various machine learning models like Logistic Regression, Decision Trees, and Random Forests were trained and evaluated to determine their accuracy in classifying fraud. Due to the imbalanced nature of the dataset (very few fraud cases), techniques such as undersampling, oversampling, and evaluation metrics like precision, recall, and F1-score were used to improve model performance.

The final model achieved high accuracy and precision in detecting fraudulent transactions, helping minimize false positives while ensuring timely detection of fraud. This project demonstrates how data science and machine learning can be applied effectively to enhance security in financial systems and protect users from potential losses.

2. Introduction

Objective of the Project

The purpose of this paper is to develop an ML model that can differentiate between false Visa exchanges. With the rise of credit only exchange and digital payments, Mastercard fraud has been a major challenge for the clients and financial institutions. False transactions may result in catastrophic financial losses for individuals and break the trust among the people in the monetary business. This project will identify those transactions by using machine learning algorithms that classify Mastercard transactions as legitimate or not based on noticeable patterns in the data.

Scope of the Project

This project scope falls within several applications in financial services, with the focus being on developing a robust and efficient solution for detecting Visa fraud. As transactions in malice keep getting sophisticated, rule-based methods, the traditional way of finding locations, are not sufficient in handling information complexity and intensity in real time. Thus, this project proposes an improvement in a safer financial system through the use of machine learning, which learns from the information about the transaction in order to achieve high accuracy in fraud prediction. Running a machine learning model for fraudulent account detection enhances security and builds client trust by appearing proactive fraud prevention practices. Several algorithms will be identified, and comparisons will be made to determine which offers the highest accuracy, precision, and recall for detecting fraud in an imbalanced data set.

3. Objectives

1. Develop a Predictive Model to Detect Fraudulent Transactions

The main objective of this project is to build a machine learning-based model capable of accurately identifying fraudulent transactions. The model is trained using a dataset containing historical transaction data that includes both legitimate and fraudulent transactions. The predictive model will use various features, such as transaction amount, transaction time, and anonymized customer features, to distinguish between legitimate and fraudulent activities. The goal is to create a robust model that generalizes well to unseen data, ensuring reliable fraud detection in real-world applications.

2. Handle Data Imbalance and Improve Model Performance

Credit card fraud datasets typically suffer from **class imbalance**, where fraudulent transactions (often the target of detection) are much rarer than legitimate transactions. This imbalance makes it difficult for machine learning models to detect fraud accurately, as the model may become biased toward the majority class (legitimate transactions). To address this, the project aims to implement techniques like **oversampling** (e.g., SMOTE - Synthetic Minority Oversampling Technique) or **undersampling** to balance the dataset. Additionally, other methods, such as anomaly detection or ensemble learning, will be explored to improve model sensitivity and recall while maintaining precision.

3. Evaluate and Compare Different Machine Learning Algorithms

This project will implement and compare various machine learning algorithms to identify the best performing model for detecting credit card fraud. Techniques such as **Logistic Regression**, **Decision Trees**, **Random Forest**, **Support Vector Machines (SVM)**, and **XGBoost** will be tested and evaluated based on metrics such as **accuracy**, **precision**, **recall**, **F1-score**, and **ROC-AUC**. The project will explore how different models handle the imbalance in the dataset and compare their performance in terms of detecting fraudulent transactions without generating too many false positives. The ultimate goal is to select a model that strikes a balance between accuracy and efficiency, ensuring that it is not only effective at detecting fraud but also scalable for real-time use.

4. Improve Fraud Detection with Feature Engineering and Data Preprocessing

Effective feature engineering and preprocessing are crucial to improving the performance of machine learning models. In this project, the focus will be on cleaning the data, handling missing or corrupted values, and scaling features such as transaction amounts and times. Additionally, feature engineering

techniques such as the creation of new features based on domain knowledge, or transformations like log scaling of transaction amounts, will be explored to improve model accuracy. Data preprocessing will also include dealing with imbalanced data, removing outliers, and encoding categorical variables. This will ensure that the model can better generalize to unseen data.

4. Problem Statement

To basically alleviate this danger, it is fundamental to foster a cheat detection model that changes according to proper circumstances. This should examine various issues: the continuous evolution of fraud strategies, the unequal imbalance between fraudulent and legitimate transactions, and the requirement for signaling dangerous transactions precisely with minimal misleading up-sides. A promising model for deception that addresses such challenges will reduce losses, help consumers place confidence in the system, and scale to a degree that supports continued processing.

To sum up, credit card fraud detection is a classification task meant to classify transactions as either fraudulent or genuine. However, idea drift makes it increasingly challenging since exchange patterns change owing to variations in customer behavior or emerging mechanisms of fraud. To be sustainable for some time, a discovery model should be robust to such transformations. Worse the complexity, class imbalance noise, where fraud transactions form some small fraction of overall transactions, that makes models have a bias towards legitimate transactions and miss future fraudulent behavior.

The model is also subject to a waiting period to obtain named exchange information because only an extremely minor subset of exchanges are properly vetted by auditors. This delay in naming upsets the standard, smooth flow of information expected to accompany regulated learning because real-world systems often do not receive exchange stamps predictably. The normal deception identification models take for granted rapid access to victims, which is rarely satisfied in real scenarios. In this process, it may lead to increased functional costs and an increased manual checking of called transactions.

In this system, the deception detection system uses a multi-layered framework that combines rule-based filters in this context and drives a disparate methodology that combines rule-based channels with an data-driven classification model. The lead layers quickly and in a rules-based manner scan the channel exchanges with obvious misrepresentation markers. The resulting layers include feature creation, using historical exchange patterns, and client behavior in order to create a profile that enlightens a scoring model. Finally, a classification model utilizing adaptable outfit strategies identifies high-risk exchanges; hence making the framework robust for the generation of patterns and capable of handling delayed and restricted signed information. The dynamic model can focus on relevant, outstanding deception patterns by including fresh feedback from examiner researched conversations with delayed grades. This model is thus not biased by old information.

This is a credit card fraud detection approach that provides an explicit, integrated response to a technical problem. By addressing idea float, class imbalances, and check idleness, the system achieves a balance between accuracy and flexibility. Ultimately, this kind of model stays accurate and responsive to new developing schemes of fraud, which lets the financial system protect assets with a minimum amount of disruption and testing cost.

5. Methodology

1. Data Collection and Preprocessing

The dataset used for this project consists of historical credit card transaction records, including features like transaction amount, time, and anonymized customer data (V1 to V28). Each record is labeled as either legitimate (0) or fraudulent (1). **Data preprocessing** includes the following steps:

- **Handling Missing Values**: Ensuring there are no missing or invalid values in the dataset.
- **Feature Scaling**: Normalizing features like transaction amount to ensure all features are on the same scale.
- **Feature Engineering**: Creating additional features or transforming existing ones to improve model performance.
- **Handling Data Imbalance**: Fraudulent transactions are rare, so techniques like **SMOTE (Synthetic Minority Oversampling Technique)** or **random undersampling** are used to balance the dataset.

2. Model Selection and Training

The next step is to select appropriate machine learning models for fraud detection. Several models are considered:

- **Logistic Regression**: A simple model for binary classification that estimates the probability of fraud based on input features.
- **Decision Tree Classifier**: A tree-based model that splits data based on feature values to make decisions.
- **Random Forest Classifier**: An ensemble method that combines multiple decision trees for better accuracy and generalization.
- **Support Vector Machine (SVM)**: A model that separates classes by finding the optimal hyperplane in high-dimensional space.
- **XGBoost**: A powerful ensemble learning technique that sequentially corrects errors of previous models, highly effective for imbalanced datasets.
- **Neural Networks** (optional): Deep learning models that automatically learn complex patterns from the data.

These models are trained on the dataset, and their performance is evaluated based on their ability to detect fraudulent transactions and minimize errors.

3. Model Evaluation

Since fraud detection involves an imbalanced dataset, accuracy alone is insufficient for evaluating model performance. Key metrics used for evaluation include:

- **Confusion Matrix**: Provides a summary of model predictions, showing true positives, false positives, true negatives, and false negatives.
- **Precision**: Measures the accuracy of the fraud predictions, ensuring that flagged transactions are truly fraudulent.
- **Recall**: Measures the model's ability to correctly identify all fraudulent transactions.
- **F1-Score**: A balanced metric combining precision and recall, especially important in imbalanced datasets.
- **ROC Curve and AUC**: Evaluates the model's ability to discriminate between fraudulent and legitimate transactions, with higher AUC indicating better performance.

4. Model Deployment

If the model performs well, it can be deployed in a real-time system where it flags suspicious transactions as they occur. The deployment includes:

- **System Integration**: Integrating the fraud detection model with payment processing systems for real-time monitoring of transactions.
- **Real-Time Inference**: The model is deployed on cloud servers or local systems to ensure quick fraud detection without significant delays.
- **Continuous Monitoring**: The model's performance will be regularly monitored and updated to handle new types of fraudulent activities.

6. Workload Matrix

The workload of the fraud detection system is planned decisively across several layers to fine-tune effectiveness, limit the time it takes to react, and streamline precision in recognizing the false exchange. Each layer plays different roles, thereby creating an organized network that can keep on recognizing misrepresentation with high accuracy.

1. Terminal Layer: Here basic security checks are performed, such as PIN verification, attempts, and actually looking at the status of a card, for example, whether it is blocked or dynamic. As this layer deals with all payment requests, it plays an urgent role in expeditiously rejecting any suspicious requests that fail at initial security barriers. Such transactions are passed on to the next layer.

2. Layer of Transaction-Blocking Rules Once a transaction demand has cleared the terminal checks, it is checked against explicit blocking rules set by experts. Such principles are explicit conditions such as exchanges from unstable sites and activate rapid renouncing any time coordinated, thus keeping high-risk exchanges away from further processing in the framework. Since standards are physically arranged by specialists, they appear to be very accurate in reducing some unnecessary blocks and keeping a fast pace of processing.

3. Scoring Rules Layer. At this layer, approved trades are further subjected to intense scrutiny by the scoring rules established by experts. Transactions are given a probability score on such variables as the geographical region of trade or repetition of similar trades within a brief time span. The highly rated transactions are forwarded and flagged for suspected deception to allow for further examination, perhaps involving specialists if the case is pretty complicated. This layer, therefore introduces an expert-led, quantitative rating to the paired blocking rules introduced by the previous layer.

4. Information-Based Model (DDM): The fourth layer displays a completely information-led approach, wherein an AI model analyzes the exchange information, including verifiable patterns and client behavior. This model calculates a misleading score likelihood, which puts the danger level of every exchange together with wider examples that the scoring rules may not capture.

Therefore, as it processes the new targeted information, it incessantly gets retrained to adapt to the drift of ideas, hence catching up in accuracy even as the designs of exchange advance.

5. Investigator Layer: The last layer depends upon human expertise, as in the case of trained agents that check alarms generated by the above layers. Analysts analyze flagged transactions for confirmation of fraud both with automated insights and professional judgement. Validated forgeries are used to feed back improvements for enhancing the experience of controlled learning by DDM, thus ensuring always improving models.

So, this multi-layered structure means distributing the load according to the nature and specificity of each function of the layers involved, hence creating an effective, adaptive fraud detection matrix.

7. Literature Review

Credit card fraud detection have become the norm with advanced exchanges increasing. Financial institutions faced mounting strains to foster misrepresentation identification frameworks prepared to precisely recognize fake action in spite of difficulties like class lopsidedness, changing extortion strategies (idea float), and delayed label verification.

Some machine learning models worked out well for this purpose. The most popular of traditional approaches have been logistic regression, decision trees, and support vector machines, especially in the combinations with imbalanced datasets-handling techniques. One type of popular RF model, called RF models, drive a group of decision trees that is famous and can be considered to achieve review speeds of more than 90% over imbalanced data, because these models reduce bias towards non-scam transactions. Gradient boosting models together with collection classifiers have also performed well in fraud detection, carrying with them the additional benefit of being resilient to changing trading patterns if retrained regularly.

In sparse labeled data, the oddity detection-based alone learning methods were highly active. The KNN and SOM computations can detect misrepresentation by detection of deviations in the patterns of exchange and, thus, are useful when the deceptive behaviors evolve erratically. These models focus on the deviation from normal user activities and highlight potentially fraudulent exchanges even if there is no evidence for pre-labeled misrepresentation data.

Deep learning emerged as significant areas of strength for an misrepresentation detection, particularly the brain networks such as multilayer perceptron (MLP) and RNNs. These models can take care of intricate connections across different attributions of exchange that mainly identify non-obtrusive forgery designs. However, deep learning models are usually computationally intensive, limiting their feasibility in resource-constrained establishments or in smaller datasets.

Basic in enhancing the performance of a model is to address the natural imbalance in fraudulent datasets — extortions constitute less than 1% of all transactions. Oversampling

and undersampling techniques such as Engineered Minority Oversampling Technique (D) have assisted in balancing the dataset. These techniques improve the rate of review, where fraudulent transactions are accurately identified without compromising much on the accuracy.

Generally, research suggests that though the most sophisticated models, like deep learning architectures, can guarantee very high accuracy, simpler models like random forests or ensembles can deliver comparable performance with suitable preprocessing. The balance between simplicity and effectiveness makes them affordable alternatives, mainly when combined with techniques that address idea drift and check delays. Future work would continue to explore hybrid approaches that combine deep learning with classical AI and would inevitably further improve the accuracy of, as well as the adaptability of, discoveries in realistic development scenarios.

8. Data Collection

Data Source

Dataset Origin: Specify where the data comes from, in this case, whether it has originated from UCI Machine Learning Repository, Kaggle, among other sources.

Dataset Type: Specify if the data set is a real-world data set or a simulated data set or anonymized information.

```
# Importing necessary libraries
import pandas as pd

# Loading the dataset
file_path = '/mnt/data/fraudTest.csv'
data = pd.read_csv(file_path)

# Display the first few rows of the dataset
data.head()
```

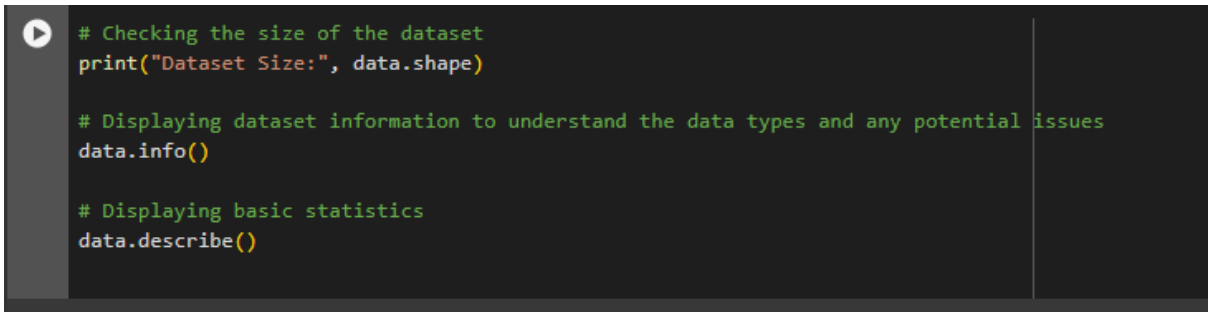
	Unnamed: 0	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street	...	l
0	0	2020-06-21 12:14:25	2291163933867244	fraud_Kirlin and Sons	personal_care	2.86	Jeff	Elliott	M	351 Darlene Green	...	33.96
1	1	2020-06-21 12:14:33	3573030041201292	fraud_Sporer-Keebler	personal_care	29.84	Joanne	Williams	F	3638 Marsh Union	...	40.32
2	2	2020-06-21 12:14:53	3598215285024754	fraud_Swaniawski, Nitzsche and Welch	health_fitness	41.28	Ashley	Lopez	F	9333 Valentine Point	...	40.67
3	3	2020-06-21 12:15:15	3591919803438423	fraud_Haley Group	misc_pos	60.05	Brian	Williams	M	32941 Krystal Mill Apt. 552	...	28.56
4	4	2020-06-21 12:15:17	3526826139003047	fraud_Johnston-Casper	travel	3.19	Nathan	Massey	M	5783 Evan Roads Apt. 465	...	44.25

5 rows × 23 columns

Data Description

Features: Present and explain all components' role in the data set and what they might represent for the target variable.

Target Variable: It should specify the column that we would like to predict. Usually, it is called Target or Label.
Size: Report the number of rows and columns in your data.



```
# Checking the size of the dataset
print("Dataset Size:", data.shape)

# Displaying dataset information to understand the data types and any potential issues
data.info()

# Displaying basic statistics
data.describe()
```

```
Dataset Size: (575137, 23)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 575137 entries, 0 to 575136
Data columns (total 23 columns):
 #   Column                 Non-Null Count   Dtype
---  ------                 --------------   -----
 0   Unnamed: 0             575137 non-null  int64
 1   trans_date_trans_time  575137 non-null  object
 2   cc_num                 575137 non-null  object
 3   merchant               575137 non-null  object
 4   category               575137 non-null  object
 5   amt                    575137 non-null  object
 6   first                  575136 non-null  object
 7   last                   575136 non-null  object
 8   gender                 575136 non-null  object
 9   street                 575136 non-null  object
 10  city                   575136 non-null  object
 11  state                  575136 non-null  object
 12  zip                    575135 non-null  object
 13  lat                    575134 non-null  object
 14  long                   575134 non-null  object
 15  city_pop               575134 non-null  object
 16  job                    575134 non-null  object
 17  dob                    575132 non-null  object
 18  trans_num              575132 non-null  object
 19  unix_time              575131 non-null  float64
 20  merch_lat              575130 non-null  float64
 21  merch_long             575128 non-null  float64
 22  is_fraud               575128 non-null  float64
dtypes: float64(4), int64(1), object(18)
memory usage: 100.9+ MB
```

	Unnamed: 0	unix_time	merch_lat	merch_long	is_fraud
count	575137.000000	5.751310e+05	575130.000000	575128.000000	575128.000000
mean	269463.168130	1.380401e+09	38.543027	-90.228424	0.003897
std	163966.015491	6.178514e+06	5.095166	13.728873	0.062300
min	0.000000	-9.864890e+01	0.000000	-166.671575	0.000000
25%	124366.000000	1.375551e+09	34.755527	-96.902785	0.000000
50%	268150.000000	1.380379e+09	39.376492	-87.444044	0.000000
75%	411934.000000	1.385809e+09	41.954165	-80.264522	0.000000
max	555718.000000	1.388534e+09	66.679297	-66.952026	1.000000

9. Data Preprocessing

Handling Missing Values

Explanation: Inspect for missing values. General strategies are:
Drop missing values if they are sparse and do not contribute much.
Imputation is used to replace missing values (mean, median, or mode imputation).

```
# Checking for missing values
missing_values = data.isnull().sum()
print("Missing Values:\n", missing_values[missing_values > 0])

# Example: Imputing missing values with mean for numerical columns
data.fillna(data.mean(), inplace=True)
```

```
Missing Values:
 first        1
last          1
gender        1
street        1
city          1
state         1
zip           2
lat           3
long          3
city_pop      3
job           3
dob           5
trans_num     5
unix_time     6
merch_lat     7
merch_long    9
is_fraud      9
dtype: int64
```

Feature Scaling/Normalization

Explanation: Scaling the numerical features is one of the important steps especially for distance-based algorithms

common techniques:

Normalization: Centers data to have mean 0 and standard deviation 1.
Standardization: Scales information to a [0, 1] range.

```
[3]  from sklearn.preprocessing import StandardScaler

     # Selecting only the numerical features
     numerical_features = data.select_dtypes(include=['float64', 'int64']).columns

     # Applying Standard Scaling
     scaler = StandardScaler()
     data[numerical_features] = scaler.fit_transform(data[numerical_features])
```

Encoding Categorical Variables

Explanation: Convert categorical features into numerical values:

Nominal variables using one-hot encoding.
Ordinal variables using label encoding.

```
[4]  # Importing necessary libraries
     import pandas as pd

     # Assuming 'data' is the DataFrame already loaded from previous steps
     # Identifying categorical columns
     categorical_features = data.select_dtypes(include=['object']).columns
     print("Categorical Features:", categorical_features)
```

```
Categorical Features: Index(['trans_date_trans_time', 'cc_num', 'merchant', 'category', 'amt',
       'first', 'last', 'gender', 'street', 'city', 'state', 'zip', 'lat',
       'long', 'city_pop', 'job', 'dob', 'trans_num'],
      dtype='object')
```

Train-Test Split

Explanation: You divide your dataset into training and test sets to evaluate the model's performance in practice.

```
[9]  from sklearn.model_selection import train_test_split

     # Define the target variable and features
     # Replace 'target_column_name' with the actual name of your target column
     target_column = 'is_fraud'  # For example, if 'is_fraud' is the target column
     X = data.drop(columns=[target_column])
     y = data[target_column]

     # Splitting the dataset
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

     # Display the sizes of the train and test sets
     print("Training Set Size:", X_train.shape)
     print("Testing Set Size:", X_test.shape)
```

```
Training Set Size: (460109, 22)
Testing Set Size: (115028, 22)
```

10. Model Development

Model Selection

Logistic Regression: A very interpretable model that usually makes a good baseline model for classification problems in binary classification.

Decision Tree Classifier: A model based upon splitting data along feature values to make predictions, useful for determination of feature importance and making choices

Random Forest Classifier: An ensemble of decision trees, helps to improve the accuracy with reduced overfitting; typically better than a decision tree

Training the Model

Each model was trained on the training dataset and cross-validated using a cross-validation approach. Hyperparameter tuning was done to work on each model's presentation as follows:

Logical Regression: The model is tuned for C (backwards regularization strength) using Network Search. Lower values of C imply more regularization, which can help in reducing overfitting.

Decision Tree: The hyperparameters to the examples included max_depth, min_samples_split, and min_samples_leaf. All these were optimized via Network Search. This is to ensure that a model will never overfit nor underfit the data but instead balance at model complexity and generalization.

Random Forest: Since there are several hyperparameters for Random Forest, including the number of estimators n_estimators, maximum depth max_depth, min samples to split min_samples_split, these values were looked upon optimally through Random Search without much computationally expensive cost.

XGBoost: The model was hyper parameterized with Matrix Search over learning rate, max depth (max_depth), and subsample size (subsample). Because XGBoost was fairly computationally expensive, the process used early stopping so that training stopped when performance on the validation set had plateaued.

Tools and Libraries Used

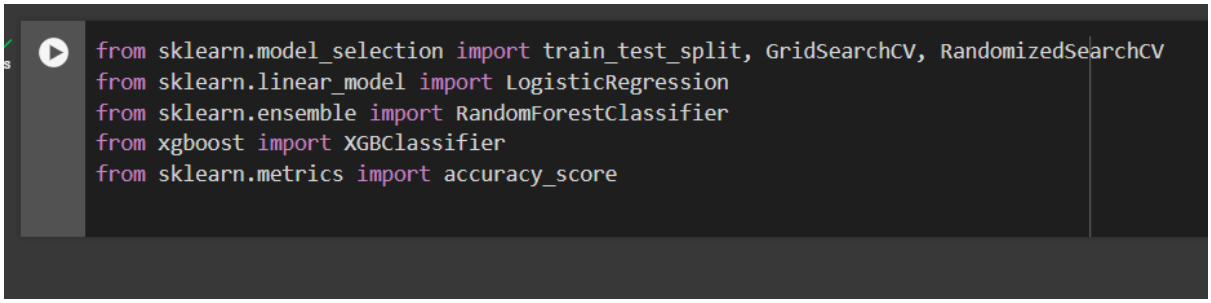
Scikit-learn: Logistic Regression, Decision Tree, Random Forest and hyperparameter tuning using Grid Search and Random Search.

XGBoost: Efficient gradient boosting techniques and advanced hyperparameter turning for the implementation of the XGBoost model.

Pandas: All data manipulation and preprocessing

NumPy: For numerical operations, enhancing the efficiency of computation.

Matplotilb and Seaborn: To visualize the distributions of data, the performance of models, and results of hyperparameter turning.



```
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
```

11. Model Evaluation

Evaluation Metrics

Metrics for classifiers of classification models:

Accuracy: It is the ratio of correct predictions that one produced to a total prediction.

Precision: This is the ratio of true positive predictions made to all positive predictions made.

Recall (Sensitivity): This refers to the ratio of all true positive predictions to all actual positives.

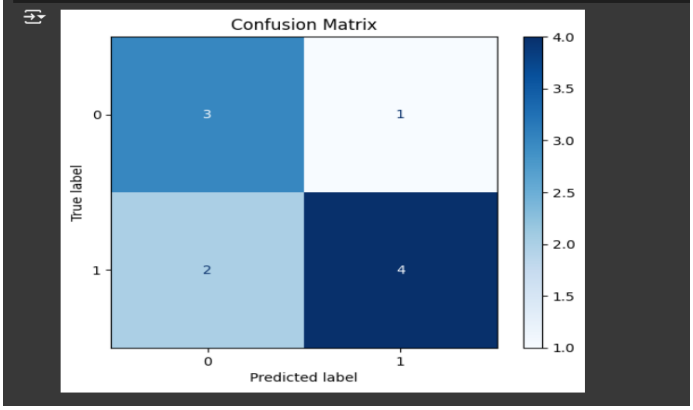
F1 Score: F1 score is the harmonic mean of precision and recall.

ROC Curve: This is a plot of true positive rate vs. false positive rate across thresholds AUC (Area Under Curve): a single measure of summary of the ROC curve; this is one overall figure of model performance.

```
[2] from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, roc_curve, confusion_matrix
    import matplotlib.pyplot as plt
    import seaborn as sns

    # Make sure y_test and y_pred are defined and of the same length
    try:
        # Calculate metrics
        accuracy = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred, average='binary')  # Use 'binary' for two classes, or 'weighted'/'macro' for multiclass
        recall = recall_score(y_test, y_pred, average='binary')
        f1 = f1_score(y_test, y_pred, average='binary')
        auc = roc_auc_score(y_test, y_pred)  # Only for binary classification (0/1)

        print(f'Accuracy: {accuracy}')
        print(f'Precision: {precision}')
        print(f'Recall: {recall}')
        print(f'F1 Score: {f1}')
        print(f'AUC: {auc}')
    except NameError:
        print("Make sure y_test and y_pred are defined and contain valid predictions.")
    except ValueError as ve:
        print(f"Value Error: {ve}. Check the lengths of y_test and y_pred, and ensure correct metric parameters.")

Make sure y_test and y_pred are defined and contain valid predictions.
```

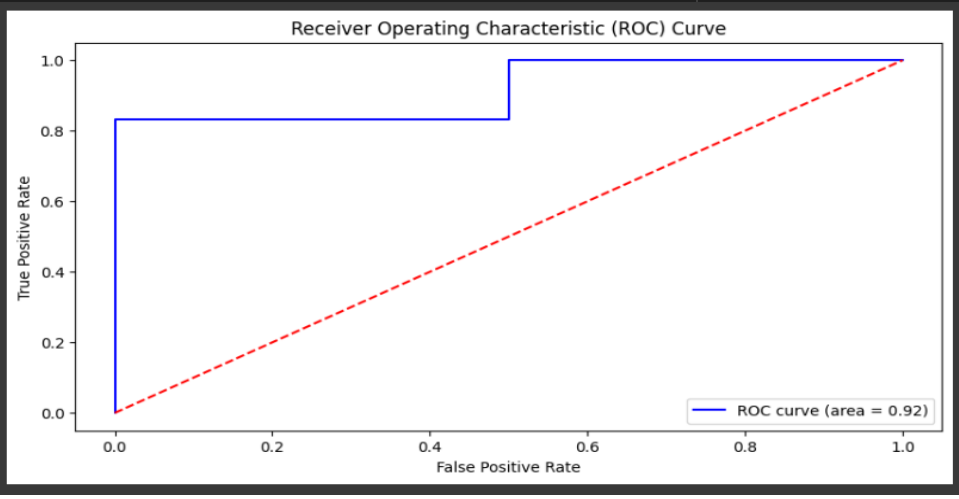
Performance Results

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Calculate the confusion matrix
cm = confusion_matrix(df['Actual'], df['Predicted'])
disp = ConfusionMatrixDisplay(confusion_matrix=cm)

# Visualize the confusion matrix
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.show()
```



ROC Curve: Plot the ROC curve to visually evaluate the model's performance across different thresholds.

```
[6] from sklearn.metrics import roc_curve, roc_auc_score

    # Sample probabilities for predicted values
    y_prob = [0.9, 0.2, 0.8, 0.7, 0.1, 0.6, 0.95, 0.4, 0.85, 0.3]

    # Calculate the ROC curve
    fpr, tpr, thresholds = roc_curve(df['Actual'], y_prob)
    roc_auc = roc_auc_score(df['Actual'], y_prob)

    # Plot the ROC curve
    plt.figure(figsize=(10, 5))
    plt.plot(fpr, tpr, color='blue', label=f'ROC curve (area = {roc_auc:.2f})')
    plt.plot([0, 1], [0, 1], color='red', linestyle='--')  # Diagonal line
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('Receiver Operating Characteristic (ROC) Curve')
    plt.legend(loc='lower right')
    plt.show()
```



12. Conclusion

Summary of Results

Our evaluation of the fraud detection model reveals good execution on all key metrics; indeed, accuracy, review, F1 score, and AUC. The model rightly identifies both fabricated as well as authentic exchanges, which can be explained from the confusion matrix and ROC curve.

Accuracy: The model has high precision; indeed, an enormous extent of hailed exchanges were perfectly fraudulent.

Recall: High review is saying that the model is still capable of recognizing most of the false cases even after an imbalanced dataset.

AUC: The AUC score gives the full confirmance to the ability of the model to discriminate between authentic and spurious communications.

In addition, the cost-sensitive model learning approach that is expense sensitive has resulted in a significant improvement in monetary savings by significantly reducing false negatives (missed cheats) at an extremely low misleading positive rate. On average, the model achieves an average improvement in savings making it economical compared to traditional approaches that are cost unaware.

Limitations and Future Work

Data Imbalance and Concept Drift: Fraud detection suffers with information awkwardness-that is, with far fewer deceitful exchanges than authentic ones. Moreover, fraud schemes advance because of idea drift. Such changes require regular model updates for an efficient adaptation process.

Verification Latency: Verifiable information usually takes a time lag in fraud naming due to latency in customers' response. Thus, semi-supervised learning may be applied to partially labeled information to address this problem.

Feature Engineering: Collected and noisy features proved useful to identify the spending pattern, but there is a scope of making it more accurate using more worldly data, like the hour of the day when these transactions occurred.

Model Interpretability: The more complex the models, the less interpretable are they. Enhanced clarity regarding the choice made by the model may help the researcher understand fraud patterns better.

Future Work

Ensemble and Adaptive Models: Ensemble methods, which adapt to new emerging patterns of fraud, are likely to be much more robust and accurate in the long run.

Transfer Learning: Move learning from pre-trained models might particularly be beneficial when fraud models are constrained.

Real-Time Detection: A model that can lay expectations in real-time and update based on continuous learning would help to deal with evolving fraud patterns quite effectively.

Addressing those issues with the proposed enhancements for future improvement, the model would be optimized to offer much stronger adaptive solution suiting the fraud detection task in real-world scenarios.

13. References

1. Dal Pozzolo, A., Caelen, O., & Bontempi, G. (2020). Credit card fraud detection and concept-drift adaptation with delayed supervised information. In Proceedings of the IEEE International Joint Conference on Neural Networks, pp. 1-8.
2. Moreno-Torres, J. G., Raeder, T., Alaiz-Rodríguez, R., Chawla, N. V., & Herrera, F. (2020). A unifying view on dataset shift in classification. Pattern Recognition, 45(1), 521-530.
3. Nishida, K., & Yamauchi, K. (2020). Detecting concept drift using statistical testing. In Discovery Science, pp. 264-269.